

A Comparison of Symbolic and Neuro-Symbolic Strategies for LTLf Runtime Monitoring

Author name

Date

Abstract

TODO

1 Introduction

Linear Temporal Logic (LTL) [23] is a logic formalism for expressing temporal properties of reactive systems. It extends traditional propositional logic with modal operators, allowing the specification of rules that must hold *through time*, and it is interpreted over traces of discrete events. Variants of LTL interpreted over finite traces have been proposed for modeling finite, but length-unbounded, process executions (see Sec. 2.2), making it suitable for finite-horizon problems that often arise in the context of AI and BPM. Temporal logics have provided one of the most popular tool in Formal Methods for verification of hardware or software system behaviors. In particular, model checking is the problem of verifying whether a model of a system meets a given specification. Despite the importance and the progress made in model checking, it is not always possible to check the model against the specification. This is due to the inherent complexity of the system or the fact that the model is only partially correct or not known. In such cases, one can resort to simpler verification approaches, such as runtime monitoring, where the system’s execution is continuously observed on-the-fly and any permanent violation or fulfillment of the specified property is immediately detected.

Standard symbolic techniques for both model checking and runtime monitoring have achieved great success, but they also suffer for some inherent limitation. Among the others there is the difficulty in dealing with the increasing growth of big data and the lack of flexibility in dealing with highly unpredictable environments, say e.g. environments with noisy sensors or probabilistic uncertainties, and adapting the specification when soft violations could happen. Conversely, pure deep learning architectures, such as Recurrent Neural Networks (RNNs) and Transformers, excel at modeling high-dimensional, noisy data distributions, but they likely generate outputs that violate critical safety constraints, as they lack a mechanism to adhere to formal logics. This naturally leads to the use of hybrid approaches, which can possibly overcome some of the mentioned limitations.

In this paper, we will compare different techniques for runtime monitoring, from purely symbolic methods to the state-of-the-art neuro-symbolic architectures. We start revisiting the neural-symbolic approach to runtime monitoring, originally proposed by [21, 22] through the RuleRunner system [20]. The authors claimed that the resulting neural monitors could exploit GPU parallelism for efficient verification, outperforming purely symbolic mechanisms. Since then, some significant developments have reshaped the landscape. First, LTL_f [6] has become the standard formalism for

temporal reasoning over finite traces in AI, bringing with it mature tools for compiling formulas into DFAs. Then, differentiable automata representations such as DeepDFA [24] have emerged, enabling the integration of temporal logic knowledge into neural architectures through gradient-based learning. We present a systematic comparison of three paradigms for LTL_f runtime monitoring: (i) purely symbolic DFA-based monitoring, (ii) rule-based neural-symbolic monitoring in the RuleRunner tradition, and (iii) automata-based neural-symbolic monitoring using DeepDFA.

TODO: Mention briefly the experimental results obtained.

2 Background

2.1 Syntax and Semantics of LTL_f

Given a finite set $\mathcal{P}$ of atomic propositions, an LTL formula φ has the form:

$$\varphi ::= \top \mid \perp \mid a \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid X\varphi \mid \varphi_1 U \varphi_2,$$

where $a \in \mathcal{P}$. We use $\top$ and $\perp$ to denote *True* and *False*, respectively. X (Next) and U (Until) are temporal operators. Other temporal operators can be derived as follows: R (Release) is defined as $\varphi_1 R \varphi_2 \equiv \neg(\neg\varphi_1 U \neg\varphi_2)$, G (globally) is defined as $G\varphi \equiv \perp R \varphi$, and F (eventually) is defined as $F\varphi \equiv \top U \varphi$. LTL_f is a finite-trace language derived from LTL and share the same grammar, with the difference that X is understood as Strong Next and require that the next time step in a given trace at a given position exists. In contrast, Weak Next is denoted by W and does not require the existence of the next time step, being interpret as $\top$ if evaluated at the last time step of the trace. It can be expressed in terms of Strong Next by $W\varphi \equiv \neg X \neg \varphi$.

A trace $\sigma = (\sigma_0, \sigma_1, \dots, \sigma_T)$ is a sequence of propositional assignments to the propositions in $\mathcal{P}$, where $\sigma_t \in 2^{\mathcal{P}}$ is the set of all and only propositions that are true at instant t . We use $|\sigma|$ to denote the length of the trace, and $\sigma[0, i]$ to denote a proper prefix of σ from time step 0 to time step i . Note that, if the propositional symbols in $\mathcal{P}$ are all *mutually exclusive*, i.e., the domain produces exactly one symbol true at each step, then we have $\sigma_t \in \mathcal{P}$. Let $last = |\sigma| - 1$. We inductively define when an LTL_f formula φ is true at an instant i (for $0 \leq i \leq last$), written $\sigma, i \models \varphi$, as follows:

- $\sigma, i \models a$ iff $a \in \sigma_i$, for $a \in \mathcal{P}$;
- $\sigma, i \models \neg\varphi$ iff $\sigma, i \not\models \varphi$;
- $\sigma, i \models \varphi_1 \wedge \varphi_2$ iff $\sigma, i \models \varphi_1$ and $\sigma, i \models \varphi_2$;
- $\sigma, i \models X\varphi$ iff $i < last$ and $\sigma, i + 1 \models \varphi$;
- $\sigma, i \models \varphi_1 U \varphi_2$ iff for some j such that $i \leq j \leq last$, we have $\sigma, j \models \varphi_2$, and for all k such that $i \leq k < j$, we have $\sigma, k \models \varphi_1$.

We say that σ satisfies ϕ , written $\sigma \models \phi$, if $\sigma, 0 \models \phi$. A formula φ is satisfiable if it is true wrt some trace σ , and is valid, if it is true wrt every trace σ . On the empty trace ε , Boolean connectives behave as usual, atoms are false, U and X are false, and R and W are true. Note that $F\top$ holds exactly on non-empty suffixes of a trace, and $G\perp$ exactly on the empty one.

In the following, we will use $sub(\varphi)$ to denote the set of proper subformulas of φ , and we will refer to it as the syntactic closure of φ . Given a formula φ and a propositional assignment σ_t at time

t , we are also interested in computing the residual formula, i.e the formula that has to be satisfied at time $t + 1$. As explained in details in Sec. 4.3, a residual formula can be computed using formula progression $\square$, which returns a positive Boolean formula on $sub(\varphi) \cup sub(F\top) \cup sub(G\perp)$. We will use $cl(\varphi)$ to denote the set of residual formulas, and we will refer to it as the residual closure of φ .

2.2 An Overview of Temporal Logics over Finite Traces

The necessity and advantage of adapting LTL from infinite to finite traces have been studied since a long time [16, 18]. This led to a prolific development of several logic languages capable to express temporally extended properties over a finite sequence of time steps. Here, we mention some of these languages that have been used for monitoring tasks.

The original RuleRunner system [20] refers to FLTL as a finite-trace variant of LTL. As already mentioned, in the last decade LTL_f [7, 8, 4] has established as one the standard language for modeling finite traces of executions. Notably, FLTL and LTL_f share essentially the same syntax and semantics, offering two next operators, with a strong next operator X which is always false at the last position of the trace since no successor exists, and a weak next operator W which is always true at the last position (see Sec. 2.1 for details). Note that FLTL makes explicit use of an *END* symbol for signaling the last element of the trace, but this can be easily expressed with the Weak Next operator by $END \equiv W\perp$.

In general, standard FLTL (and LTL_f) is not defined for the empty trace. [9] proposed $LTL^\mp$, a formalism composed by two versions of LTL: in the weak view (LTL^-), every formula is satisfied by the empty word, while in the strong view (LTL^+), the empty word does not satisfy any formula. This means that the same formula can have a different truth value in the two views. Using this approach, one needs the strong and weak view to be entwined to establish a sound meaning in the context of negation.

Another way of looking at finite traces is to consider them as prefixes of unknown infinite traces. Building on this idea, [1, 2] proposed LTL_3 , which uses the same syntax of LTL_f but a different semantics, evaluating each formula to one of the truth values $\{\top, \perp, ?\}$, with $\top$ and $\perp$ being complementary to each other and $?$ being complementary to itself. Specifically if every infinite trace with prefix σ evaluates φ to the same truth value $\top$ or $\perp$, then LTL_3 also evaluates φ to this truth value, otherwise it evaluates to $?$. In contrast with the other formalisms described in this section, the semantics of LTL_3 cannot be defined inductively on the structure of the formula.

Some temporal logics have been proposed as direct extensions or modification of LTL_f . LDL_f (Linear Dynamic Logic over finite traces) [7, 8] is an extension of LTL_f to increase the expressivity to regular languages, built as a finite-trace adaptation of the infinite-trace language LDL (Linear Dynamic Logic) [25] and inspired by Propositional Dynamic Logic [10, 12]. It has a "dynamic operator" $\langle \rho \rangle \varphi$, which intuitively states that, from the current instant, there exists an execution satisfying the RE_f (Regular Expression over finite traces) [13, 15] ρ such that its last instant satisfies φ . Other formalisms are $PPLTL_f$ (Pure-Past LTL_f) [5], to refer to past events instead of future ones, and LTL_f^{MT} (LTL_f Modulo Theory) [11], where proposition letters are replaced with first-order formulas interpreted over arbitrary theories.

TODO: Mention RVLTL, RLTL, TRLTL.

2.3 Symbolic Monitoring

Deterministic Finite Automata. Any LTL_f formula φ can be translated into a Deterministic Finite Automaton (DFA) [6] $A_\varphi = (2^{\mathcal{P}}, Q, q_0, \delta, F)$, where $2^{\mathcal{P}}$ is the automaton alphabet, Q is the finite set of states, $q_0 \in Q$ is the initial state, $\delta : Q \times 2^{\mathcal{P}} \rightarrow Q$ is the transition function, and $F \subseteq Q$ is the set of final states. We also define the extended transition function over traces $\delta^* : Q \times (2^{\mathcal{P}})^* \rightarrow Q$ recursively as:

$$\begin{aligned} \delta^*(q, \epsilon) &= q \\ \delta^*(q, \sigma + \mathbf{x}) &= \delta^*(\delta(q, \sigma), \mathbf{x}), \end{aligned} \tag{1}$$

where $\sigma \in 2^{\mathcal{P}}$ is a symbol and $\mathbf{x} \in (2^{\mathcal{P}})^*$ is a trace. The automaton accepts the trace σ iff $\delta^*(q_0, \sigma) \in F$, and in that case we say that σ belongs to the language of the automaton, denoted as $\mathcal{L}(A_\varphi)$. We have that φ and A_φ are equivalent because, for any trace $\sigma \in (2^{\mathcal{P}})^*$:

$$\sigma \in \mathcal{L}(A_\varphi) \iff \sigma \models \varphi. \tag{2}$$

We can rely on the use of DFAs for several reasoning tasks, like model checking, planning and synthesis. In particular, monitoring formula φ becomes a deterministic walk over the automaton A_φ : the monitor keeps a single current state q , initialized to q_0 , and on observing the time point $\sigma_t \in 2^{\mathcal{P}}$ it advances $q \leftarrow \delta(q, \sigma_t)$. Because the automaton is fixed and deterministic, each step costs a single transition lookup, independent of the trace seen so far and of the size of φ . In the following, DFA-based monitoring will serve as the exact baseline against which the neuro-symbolic paradigms will be measured.

Verdicts and early termination. To address online monitoring, we can resort to a three-valued runtime semantics [3]: at each step the monitor emits SATISFY, VIOLATE, or UNDECIDED, the last meaning that the observed prefix can still be extended into both an accepting and a non-accepting continuation. A definite verdict can be issued as soon as the outcome is settled regardless of the future, which corresponds to two structural properties of the current state, both precomputed once by backward reachability over A_φ . A state is a *trap* if no accepting state is reachable from it: reaching a trap means φ is permanently violated, so the monitor halts with VIOLATE. Dually, a state is an *accepting sink* if every state reachable from it (including itself) is accepting: the monitor halts with SATISFY. With trap and sink sets materialized at compile time, the per-step verdict reduces to two set-membership tests on top of the transition lookup. As long as the current state is neither, the verdict is UNDECIDED and monitoring continues. Some specifications never reach a trap or an accepting sink, so STEP returns UNDECIDED for the entire trace. An example is the response pattern $G(a \rightarrow Fb)$, which can always be satisfied by a later b and always be rescued from violation, hence has neither an accepting sink nor a trap. For these the verdict is emitted at the end of the trace, and the monitor reports it by testing acceptance of the state reached, i.e. SATISFY iff $\delta^*(q_0, \sigma) \in F$ and VIOLATE otherwise. Note that, in online monitoring, we require the monitor to be prefix-sound, i.e. if it outputs SATISFY (resp. VIOLATE) after reading the prefix $\sigma[0, t]$, then $\sigma' \models \theta$ (resp. $\sigma' \not\models \theta$) for every trace σ' sharing the same prefix. This is a stronger property than being able to emit the correct verdict at the end of the trace.

3 Automata-based Neuro-Symbolic Monitoring: DeepDFA

3.1 Architecture

DeepDFA [24] keeps the automaton of the symbolic paradigm but replaces its explicit state walk with a differentiable, tensorized one; we adopt the same forward formulation used for predictive monitoring of business processes by [19]. In its original formulation DeepDFA is a *probabilistic relaxation* of a DFA: a probabilistic finite automaton whose transition logits and acceptance logits are real-valued and *learnable*. Temperature-scaled softmax constrains each transition row to the probability simplex, temperature-scaled sigmoid produces the acceptance vector, and annealing drives both toward the one-hot/Boolean limit. This is distinct from the Gumbel–Softmax estimator used by [19] to sample differentiable near-symbolic *inputs* for a fixed automaton [14, 17]. Accordingly, starting from the minimal DFA $A_\varphi = (2^{\mathcal{P}}, Q, q_0, \delta, F)$ of Section 2.1, the general object is a three-way tensor $T \in [0, 1]^{|Q| \times |2^{\mathcal{P}}| \times |Q|}$ that is row-stochastic in its last index, $\sum_{q'} T[q, \sigma, q'] = 1$, together with an acceptance vector $v_o \in [0, 1]^{|Q|}$. When the automaton is instead *compiled* from a known LTL_f formula — the setting of this paper’s monitoring experiments, and the fixed knowledge layer of [19] — these quantities are fixed to their exact one-hot values, $T \in \{0, 1\}^{|Q| \times |2^{\mathcal{P}}| \times |Q|}$ with $T[q, \sigma, q'] = 1$ iff $\delta(q, \sigma) = q'$, and $v_o \in \{0, 1\}^{|Q|}$ with $v_o[q] = 1$ iff $q \in F$. Learning them is unnecessary for monitoring a known specification; a separate real-valued parameterization would instead be needed for the specification-adaptation problem deferred to Section 7. In either case the monitor state is no longer a single automaton state but a row vector $\mathbf{q}_t \in [0, 1]^{|Q|}$ over states, initialized to the one-hot vector $\mathbf{q}_0 = v_i$ of q_0 . Writing $T_\sigma = T[\cdot, \sigma, \cdot] \in [0, 1]^{|Q| \times |Q|}$ for the (row-stochastic) transition matrix of symbol σ , a step is a single vector–matrix product

$$\mathbf{q}_t = \mathbf{q}_{t-1} T_{\sigma_t}, \quad \text{acc}_t = \mathbf{q}_t \cdot v_o, \quad (3)$$

where $\text{acc}_t \in [0, 1]$ is the acceptance score of the prefix read so far. The following elementary invariant records exactly what is preserved by the tensorization.

Proposition 1 (Crisp equivalence). *Let $\sigma_{1:t} = \sigma_1 \cdots \sigma_t$ be a crisp prefix and let $\mathbf{q}_t$ be obtained from Equation (3) using the compiled one-hot tensor. Then*

$$\mathbf{q}_t = e_{\delta^*(q_0, \sigma_{1:t})},$$

where e_q is the indicator vector of state q . Consequently the fixed DeepDFA monitor and the symbolic DFA visit the same state after every prefix and return identical early and final verdicts.

3.2 Soft Observations and Differentiability

TODO

3.3 The Alphabet Axis

TODO

4 Rule-based Neuro-Symbolic Monitoring: RuleRunner

4.1 Architecture

A RuleRunner system is a tuple $\langle S, R_E, R_R \rangle$, where S is the state of the system, R_E is the set of *evaluation rules*, and R_R is the set of *reactivation rules*. In details:

- The state S contains observations, active rules, and truth evaluations. $R[\phi]$ denotes an active rule, while $[\phi]V$ denotes a 3-values truth evaluation, with $V \in \{T, F, ?\}$.
- Evaluation rules are used to compute the truth value of a formula ϕ under verification, given the truth values of its direct subformulas ψ_i . They have the form $R[\phi], [\psi_1]V, \dots, [\psi_n]V \rightarrow [\phi]V$. For instance, $R[a \vee b], [a]T \rightarrow [a \vee b]T$, because the disjunction is evaluated to *true* if one of the two disjuncts is true.
- Reactivation rules are used when a formula ϕ is evaluated to undecided, i.e. $[\phi]?$, in which case that formula and its subformulas are scheduled to be monitored again at the next time point of the trace. They have the form $[\phi]? \rightarrow R[\phi], R[\psi_1], \dots, R[\psi_n]$. For instance, $[\Diamond b]? \rightarrow R[b], R[\Diamond b]$, because if $\Diamond b$ is undecided, we need to re-evaluate b and $\Diamond b$ at the next step.

Both active rules and truth evaluations can be enriched by a *qualifier*, such as B (binary), L (left) and R (right), which provides additional information recording which operands within a formula/subformula are still being watched. For instance, $R[a \wedge \Diamond b], [a]T \rightarrow [a \wedge \Diamond b]?R$, because if the left conjunct a is true, it is still needed to check the truth value right conjunct $\Diamond b$ to assign a truth value to the conjunction.

Verification of a trace against a formula proceeds iteratively. At the beginning of each iteration (corresponding to the monitoring of a time point), the state S contains a set of rule names corresponding to the subformulas to be checked at that time point. First, the procedure includes observations of the current time point in the state S and fires the evaluation rules repeatedly incrementing the state itself until it reaches a fixpoint. In practice, this means computing the truth values of all subformulas $sub(\phi)$ of ϕ in a bottom-up way, from simple atoms to ϕ itself (see Figure ...). If the root evaluates to true (resp. false), the monitor halts with verdict SATISFY (resp. VIOLATE). Otherwise, the reactivation rules are fired, and the result constitute the state of the next iteration. Note that in this case the state is not expanded, but replaced by the result of reactivation rules. This is used to flush all the previous truth evaluation, which are to be computed from scratch at the new time point. The procedure terminates with a decided verdict because of special END-triggered evaluation rules, that returns either true or false if we consumed the whole trace. A RuleRunner system can be translated in a logic program, which in turn can be encoded in a Recurrent Neural Network with a single hidden layer, s.t. each evaluation and reactivation rule is a mapping from the input layer to the output one. For a detailed description of RuleRunner and verification algorithm see [21, 20].

The rule system underlies two behaviorally-equivalent neural encodings, both of which we evaluate in Section 6. In the *flat* encoding [21], the whole rule set is compiled into a single recurrent network following the standard CILP translation $\square$: one hidden unit per rule, weights realizing each Horn clause, and the fixpoint of the evaluation phase recovered by iterating the forward pass until the fixpoint is met. In the *structured* encoding [22], the same rules are instead grouped as one CILP subnetwork per parse-tree node, wired according to the syntax of φ . Operationally, we execute one bottom-up sweep of the evaluation modules and then fire every reactivation module

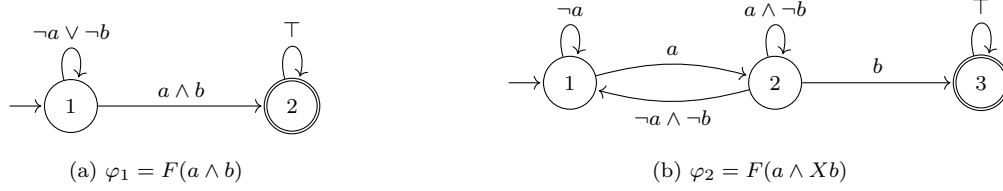


Figure 1: Minimal DFAs for φ_1 and φ_2 .

independently, taking the union of their outputs as the recurrent state of the next cell. This is functionally equivalent to horizontally composing the modules into the single recurrent network of Figure 5 in [22]. The modular form could be the substrate for *local learning*: because parameters are attached to syntactically-local subnetworks, adaptation can be confined to the subtree responsible for a misclassification rather than diffused across a monolithic network. This is left for future work.

The Nested-Temporal Limitation. RuleRunner’s defining design choice is to maintain *a single shared state, with exactly one truth register per syntactic subformula*. This is what gives it a constant, compile-time rule count, a fixed-size network, and a non-branching state, and it is well suited to many flat temporal specifications. The problem is that the architecture *assumes each subformula has exactly one live truth value per time point*, and this assumption is not always true. There are some formulas with nested temporal operators for which the outer operator spawns a fresh instance of its argument while a prior instance of the *same* subformula is still resolving, and the two distinct live constraints are conflated onto the one shared object.

Consider the two formulas $\varphi_1 = F(a \wedge b)$ and $\varphi_2 = F(a \wedge Xb)$, which differ only by a single next operator, monitored over the same trace $\sigma = (\{a\}, \emptyset, \{b\})$. Neither formula is satisfied: φ_1 would require a time point where a and b hold *together* (but a occurs only at $t = 0$ and b only at $t = 2$), and φ_2 would require a time point t with $a \in \sigma_t$ and $b \in \sigma_{t+1}$ (but the only a is at $t = 0$, and is not followed by a b at $t = 1$). The correct verdict in both cases is therefore VIOLATE, but RuleRunner can provide the correct answer only for the first formula. In contrast, DFAs have no such limitation: Figure 1 shows the DFAs for the two formulas; in both cases the verdict is VIOLATE because reading the trace over them does not lead to a terminal state.

To look at how the RuleRunner computation proceeds, first consider φ_1 . The run is:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$[a]T, [b]F \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$	$\rightarrow R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
1	$\emptyset$	$[a]F, [b]F \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$	$\rightarrow R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
2	$\{b\}$	$[a]F, [b]T \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$ $[F(a \wedge b)]? \rightarrow_{\text{END}} [F(a \wedge b)]F$	$\rightarrow \text{VIOLATE}$

Now, consider φ_2 . In this case, the run diverges:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$R[Xb] \rightarrow [Xb]?$	$\rightarrow R[Xb]M, R[b]$
		$[a]T, [Xb]? \rightarrow [a \wedge Xb]?R$	$\rightarrow R[a \wedge Xb]R$
		$[a \wedge Xb]?R \rightarrow [F(a \wedge Xb)]?$	$\rightarrow R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
1	$\emptyset$	$R[Xb] \rightarrow [Xb]?$	$\rightarrow R[Xb]M, R[b]$
		$R[Xb]M, [b]F \rightarrow [Xb]F$	
		$R[a \wedge Xb]R, [Xb]? \rightarrow [a \wedge Xb]?R$	$\rightarrow R[a \wedge Xb]R$
		$R[a \wedge Xb]R, [Xb]F \rightarrow [a \wedge Xb]F$	
		$R[a \wedge Xb], [a]F \rightarrow [a \wedge Xb]F$	
2	$\{b\}$	$[a \wedge Xb]?R \rightarrow [F(a \wedge Xb)]?$	$\rightarrow R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
		$R[Xb] \rightarrow [Xb]?$	
		$R[Xb]M, [b]T \rightarrow [Xb]T$	
		$R[a \wedge Xb]R, [Xb]T \rightarrow [a \wedge Xb]T$	
		$R[a \wedge Xb], [a]F \rightarrow [a \wedge Xb]F$	
		$[a \wedge Xb]T \rightarrow [F(a \wedge Xb)]T$	$\rightarrow \text{SATISFY}$

The divergence stems from RuleRunner’s single shared state, which keeps exactly one truth register per syntactic subformula. The decisive step is the F reactivation. At every time point, an F that remains undecided reinstalls a fresh $R[Xb]$ that will defer again, while the previous time point’s deferral is still resolving via $R[Xb]M$. As mentioned previously, the state is a single flat set of truth evaluations and rule names with no position index. A reinstalling operator can place, into that one set, a freshly-deferred instance of a subformula while a prior instance of the same subformula is still resolving. The two instances are indistinguishable, and the next evaluation pass conflates them, possibly holding contradictory values for one subformula at once.

This limitation does not depend on the encoding of RuleRunner or on the implementation artifacts. The conflation cannot be repaired at the output layer. Consider φ_2 again, the two traces $\sigma_A = (\{a\}, \emptyset, \{b\})$ (ground truth VIOLATE) and $\sigma_B = (\emptyset, \{a\}, \{b\})$ (ground truth SATISFY) for φ_2 . After the second time point both present the *identical* conflicted register $[Xb] = \{F, ?\}$, yet the correct resolutions are opposite: σ_A needs the F to win, σ_B needs the $?$ to win. Any monitor whose decision is a function of the shared register alone must answer identically on both, so no priority order over the register can be correct on both traces.

A correct fragment. The counterexample identifies the mechanism of failure, but not a syntactic boundary. Temporal nesting alone is neither necessary nor sufficient for observing the failure presented above. For example, it is easy to see that the formula $X(Xa)$ is handled correctly. Conversely, $(Xa) \wedge X(Xa)$ shares the subformula Xa between two temporal offsets, and the issue arises again: on the trace $(\emptyset, \{a\})$, the original RuleRunner returns SATISFY, whereas the correct verdict is VIOLATE. Note also that the equivalent rewriting $X(a \wedge Xa)$ is certified correct. This makes complicated to identify the maximal syntactic fragment of LTL_f for which RuleRunner is correct. Still, we can provide a readable fragment with the following grammars:

$$\begin{aligned} \pi &::= \top \mid \perp \mid p \mid \neg\pi \mid \pi \wedge \pi \\ \theta &::= \pi \mid X\pi \mid W\pi \mid \pi \mathcal{U} \pi \mid \neg\theta \mid \theta \wedge \theta \end{aligned}$$

where $p \in \mathcal{P}$. We denote the set of formulas generated by this grammar as $\mathcal{F}_{RR}$. Note that θ cannot contain nested temporal operators, i.e., the argument of X , W and U is always propositional. The following theorem shows that RuleRunner is correct for all formulas in $\mathcal{F}_{RR}$.

Theorem 1. *Let θ be a formula in $\mathcal{F}_{RR}$ and σ a non-empty finite trace. Let RR_θ be the deterministic finite-state monitor induced by RuleRunner rules. Then RR_θ returns SATISFY iff $\sigma \models \theta$. Moreover, RR_θ is prefix-sound.*

In general, if we want to check whether an LTL_f formula φ can be handled correctly by RuleRunner, we need a semantic certifier. Let RR_φ be the deterministic finite-state monitor induced by the original RuleRunner rules, and let A_φ be a canonical DFA for φ . The class of formulas for which RuleRunner is correct is

$$\mathcal{C}_{RR} = \{\varphi \mid \mathcal{L}(RR_\varphi) = \mathcal{L}(A_\varphi)\}$$

Membership is decidable exactly. Both monitors have finite state, so we can perform a search over their synchronous product. A reachable product state whose two components disagree on final acceptance yields a counterexample; breadth-first exploration yields a shortest one.

Corollary 1. *If $\varphi \in \mathcal{F}_{RR}$, then $\varphi \in \mathcal{C}_{RR}$.*

4.2 Bounded-Horizon Formulas

Now we know that RuleRunner can fail in returning the right verdict and it is certainly correct only for a fragment of LTL_f formulas. In Section 4.3 we will see how to modify RuleRunner in order to fix this problem and how much it costs. Before doing it, we want to investigate if there exists a way to enlarge the fragment of formulas we identified and still use the same RuleRunner construction, maintaining all its advantages. Inspired by [?]¹, we define bounded-horizon formulas and we present an encoding such that RuleRunner can handle this kind of formulas despite natively it cannot.

Bounded Events and Derived Traces. Consider an LTL_f formula φ . We define the future horizon of φ recursively as follows:

$$\begin{aligned} h(p) &= 0 \\ h(\neg\varphi) &= h(\varphi) \\ h(\varphi_1 \circ \varphi_2) &= \max(h(\varphi_1), h(\varphi_2)) \\ h(X\varphi) = h(W\varphi) &= 1 + h(\varphi) \\ h(G\varphi) = h(F\varphi) &= \infty \\ h(\varphi_1 U \varphi_2) = h(\varphi_1 R \varphi_2) &= \infty \end{aligned}$$

where $\circ \in \{\wedge, \vee, \rightarrow\}$. We say that if φ has a bounded horizon if $h(\varphi) < \infty$.

Now, let $\varphi_1, \dots, \varphi_m$ be the maximal non-atomic bounded subformulas of φ . The maximal horizon of φ is $H = \max_i h(\varphi_i)$. We can replace each distinct φ_i with a fresh event proposition e_i , obtaining a *RuleRunner skeleton* $sk(\varphi)$. This allow us to identify a new fragment of LTL_f formulas defined by

$$\mathcal{F}_{BE} = \{\varphi \mid sk(\varphi) \in \mathcal{F}_{RR}\}.$$

¹Their construction supports interval-indexed Until and uses strong/weak truncated-path contexts together with an extrapolation circuit to recover the Boolean value under their finite truncated-path semantics. We do not use interval-indexed Until because it would change the standard syntax and semantics of LTL_f , and it can always be rewritten using only the standard syntax, providing no real advantage but succinctness.

It is immediate to see that $\mathcal{F}_{RR} \subsetneq \mathcal{F}_{BE}$. Here are some examples of formulas that are in $\mathcal{F}_{BE}$:

formula	bounded event	skeleton
$G(Xa)$	$e \equiv Xa$	Ge
$F(a \wedge Xb)$	$e \equiv a \wedge Xb$	Fe
$G(a \rightarrow Xb)$	$e \equiv a \rightarrow Xb$	Ge
$aU(b \wedge Xc)$	$e \equiv b \wedge Xc$	aUe

To evaluate a skeleton, we turn the original trace into a trace over $\mathcal{P} \cup \{e_1, \dots, e_m\}$. Given a non-empty trace σ , we define its derived trace $\hat{\sigma}$, of the same length, by

$$\hat{\sigma}_t = \sigma_t \cup \{e_i \mid \sigma, t \models \varphi_i\}, \quad 0 \leq t \leq \text{last}.$$

Intuitively, at every time step, the derived trace adds e_i if the corresponding bounded subformula φ_i is satisfied at that time step. The key observation is that these flags are computable from a fixed window. A formula φ_i with $h(\varphi_i) = h_i$ looks at most h_i steps ahead, so its value at position t is determined by $\sigma_t, \dots, \sigma_{t+h_i}$, or by $\sigma_t, \dots, \sigma_{\text{last}}$ when the trace ends first.

Lemma 1. *Let φ be a formula with $h(\varphi) = h < \infty$. Let $l = \min(t + h, \text{last})$ and $w = \sigma[t..l]$. For every non-empty trace σ and every position $t \leq \text{last}$,*

$$\sigma, t \models \varphi \text{ if and only if } w, 0 \models \varphi$$

Now, we can use a streaming transducer with a buffer of $H + 1$ as follows: while the trace is being read, the transducer appends each observation to the buffer; as soon as the buffer holds $H + 1$ observations, the transducer evaluates the subformulas on the oldest buffered position $t - H$, emits $\hat{\sigma}_{t-H}$, and drops σ_{t-H} . This is repeated until the trace ends, when the whole derived trace is obtained. This can be viewed as a fixed pre-processing step for the ordinary RuleRunner computation, so that no rule needs to be added or modified.

Theorem 2. *Let $\alpha = sk(\varphi)$. If RR_α is language-equivalent to A_α , then for every finite trace σ the composition of the bounded-event transducer and RR_α returns SATISFY iff $\sigma \models \varphi$. Moreover, if RR_α is prefix-sound, then so is the composition.*

Corollary 2. *For every $\varphi \in \mathcal{F}_{BE}$, the bounded-event RuleRunner returns the correct final verdict, and it is prefix-sound.*

Neural Realization. The described pipeline can be compiled into a network of the same kind as standard RuleRunner's. Since the events are handled as ordinary observations, the flat and the structured CILP realizations of Section 4.1 are kept unchanged on $\hat{\sigma}$. The only differences are two new components, the buffer and the event circuit.

The buffer stores the H observations that the event circuit needs. It is a CILP-like recurrent network of the same kind as the others. In the input and output layers we have one unit per atom, and one unit per pair (p, k) , with $p \in \mathcal{P}$ and $0 \leq k < H$, recording whether p held $k + 1$ cells ago. Its rules are copies, such that the unit of p in the current observation is copied into the unit of $(p, 0)$, and the unit of $(p, k - 1)$ into the unit of (p, k) . In this way, the buffer maintains a sliding window, where the oldest observation is dropped, and the current observation enters at age 0. In this case we need no iteration to a fixpoint. As always, the pair units are then fed back as the layer's input at the next time point.

The event circuit evaluates the bounded subformulas over the window. For every parse-tree node ψ of some φ_i and every offset k of the window we introduce the two registers $[\psi, k]T$ and $[\psi, k]F$, together with rules that mirror the operator of ψ : a Boolean connective combines the registers of its children at the same offset k , while $X\psi'$ and $W\psi'$ at offset k read the registers of ψ' at offset $k + 1$. The registers of the atoms are clamped from the buffer, and the registers of each φ_i at offset 0 are the event flags of the emitted time point. Note that the whole window is available at once, so every node receives a definite value within the same time point and nothing is ever deferred. Note also that the offset k is instead an explicit address, so two occurrences of the same subformula at different offsets are different registers. This is the distinction that the single shared register of Section ?? cannot make.

As for the original RuleRunner, these rules can be grouped in two ways. The flat realization pools them into a single layer and iterates it to a fixpoint; the structured realization owns one layer per pair (ψ, k) and fires them bottom-up in the parse tree. The result is still a fixed recurrent network, whose size depends only on φ and H .

Exact Online Extrapolation. The theorem gives exact final verdicts and prefix-soundness, but it says nothing about when a verdict is emitted, and the delayed evaluation can be arbitrarily late. Consider $\varphi = G(Xa)$, which is unsatisfiable on every non-empty trace. The run over A_φ ends up in a trap state after one cell. The skeleton of the formula is Ge with $e \equiv Xa$, and the trace in which e holds at every position satisfies Ge ; the skeleton therefore stays undecided and reports VIOLATE only at the end of the trace (since e is false at the last position of every possible trace by the semantics of Strong Next). Thus, the delay is not bounded by H , but the length of the trace.

TODO

Theorem 3. *After every non-empty prefix, the extrapolated bounded-event monitor returns a three-valued label identical to that of the canonical DFA for the original formula.*

4.3 Progression-Based Reactivation

The counterexample of Section ?? should not be read as a failure of the RuleRunner idea as a whole. It isolates a more precise problem: the original construction carries information across time using registers addressed only by the syntactic subformulas of the input formula. The local evaluation rules are not the source of the error. Given the values of the direct subformulas at one time point, the extended truth tables compute the intended three-valued result. What fails is the reactivation phase: when the same subformula is reinstalled from different temporal contexts, the distinct residual meanings of that subformula are written into the same register.

The repair we consider keeps the two ingredients that make RuleRunner distinctive: a rule-based evaluation/reactivation cycle, and the possibility of translating the resulting rule system into a recurrent CILP-style network. The change is in what is carried across time. Instead of carrying only truth registers for subformulas of the original input formula, the repaired monitor carries residual formulas obtained by progression $[?, ?]$. The monitor still uses formula registers, and these registers are still associated with syntactic formulas; however, the relevant formulas are now taken from the residual closure of the input formula, not only from its original parse tree.

RuleRunner, fixed. Formula progression gives a principled account of what remains to be checked after reading one observation. Given φ and $s \in 2^{\mathcal{P}}$, let $\text{prog}(\varphi, s)$ denote the formula

that has to hold on the remaining suffix after s has been consumed, such that for every finite continuation σ , the following holds:

$$s \cdot \sigma \models \varphi \iff \sigma \models \text{prog}(\varphi, s)$$

Formally, formula progression is defined recursively as follows:

$$\begin{aligned} \text{prog}(a, s) &= \top \text{ if } a \in s, \text{ and } \perp \text{ otherwise,} \\ \text{prog}(\neg a, s) &= \perp \text{ if } a \in s, \text{ and } \top \text{ otherwise,} \\ \text{prog}(\varphi_1 \wedge \varphi_2, s) &= \text{prog}(\varphi_1, s) \wedge \text{prog}(\varphi_2, s), \\ \text{prog}(\varphi_1 \vee \varphi_2, s) &= \text{prog}(\varphi_1, s) \vee \text{prog}(\varphi_2, s), \\ \text{prog}(X\varphi, s) &= \varphi \wedge F\top, \\ \text{prog}(W\varphi, s) &= \varphi \vee G\perp, \\ \text{prog}(\varphi_1 U \varphi_2, s) &= \text{prog}(\varphi_2, s) \vee (\text{prog}(\varphi_1, s) \wedge (\varphi_1 U \varphi_2)), \\ \text{prog}(\varphi_1 R \varphi_2, s) &= \text{prog}(\varphi_2, s) \wedge (\text{prog}(\varphi_1, s) \vee (\varphi_1 R \varphi_2)). \end{aligned}$$

The repaired monitor carries a *residual formula* $\rho_t \in cl(\varphi)$, i.e. the formula that the remaining suffix starting at time t must satisfy. It is initialized to $\rho_0 = \varphi$ and updated by progression:

$$\rho_{t+1} = \text{nf}(\text{prog}(\rho_t, \sigma_t))$$

where nf is a semantics-preserving Boolean normalization discussed below. The verdict is read off the residual itself: $\rho_t \equiv \top$ every continuation satisfies φ , and the monitor halts with SATISFY; $\rho_t \equiv \perp$ no continuation does, and it halts with VIOLATE; otherwise the verdict is UNDECIDED.

As in the original system, evaluating a formula at a time point is a bottom-up computation over a parse tree, with the only difference that here we consider the subformulas of the residual instead of φ . In detail, the RuleRunner state S_t at time t holds the observations of σ_t , the active rules $R[\psi]$, and the truth evaluations $[\psi]V$ for $\psi \in \text{sub}(\rho_t) \cup \{\rho_t\}$. What is carried across time is the residual alone. Note that the undecided subformulas $[\psi]?$ of ρ_t are not reactivated as before, but flushed as every other truth evaluation: their pending contribution is already recorded in ρ_{t+1} .

The residual is carried not as a monolithic formula but in *factored* form, as the set of its top-level conjuncts:

$$\Sigma_t = \text{split}_\wedge(\rho_t)$$

where $\text{split}_\wedge(\top) = \emptyset$, $\text{split}_\wedge(\chi_1 \wedge \chi_2) = \text{split}_\wedge(\chi_1) \cup \text{split}_\wedge(\chi_2)$, and any formula whose top-level operator is not a conjunction is kept as a single element. We call *roots* of a residual the elements in Σ_t . Since progression distributes over conjunction, the elements of Σ_t can be progressed independently and the results re-collected if needed; the conjunction of the collected roots is equivalent to ρ_{t+1} above. The factorization reduces the size of the recurrent state. If we carry the whole residual, the monitor needs one register per reachable residual, i.e., a one-hot vector over $cl(\varphi)$. Let $\mathcal{R}_\varphi = \bigcup_{\rho \in cl(\varphi)} \text{split}_\wedge(\rho)$ be the residual-root vocabulary. Carrying conjuncts instead gives a *multi-hot* vector over $\mathcal{R}_\varphi$, and the gap can be exponential. For $\varphi = Fa_1 \wedge \dots \wedge Fa_n$, the reachable residuals are all subsets of still-pending conjuncts, so $|cl(\varphi)| = 2^n$, whereas $|\mathcal{R}_\varphi| = n$. Note that this saving concerns the recurrent state only: exact early verdicts still require the reachable sets of roots to be classified, as discussed below.

To summarize, the RuleRunner two-phase schema is repaired as follows:

1. **Evaluation.** A new observation σ_t is added into S_t and the evaluation rules R_E are fired bottom-up, separately for every root $\chi \in \Sigma_t$, i.e., over $\text{sub}(\chi) \cup \{\chi\}$

2. **Reactivation.** Each active root is progressed independently against σ_t . Each result is normalized and split into top-level conjuncts, and the emitted roots are conjuncted into the next state:

$$\Sigma_{t+1} = \bigcup_{\chi \in \Sigma_t} \text{split}_{\wedge}(\text{nf}(\text{prog}(\chi, \sigma_t))), \quad \rho_{t+1} = \bigwedge_{\chi \in \Sigma_{t+1}} \chi.$$

Note both phases can operate on the set of roots directly, without the need to reconstruct the residual ρ_{t+1} at every step.

Finally, the normalization nf is what keeps the construction finite. This is not required for semantic soundness, but, if we don't use it, raw syntax may grow without bound. For instance, repeated failure of ψ in $F\psi$ would produce $\perp \vee F\psi$, then $\perp \vee (\perp \vee F\psi)$, and so on.. nf treats temporal subformulas as opaque atoms and simplifies the Boolean structure (e.g., flattening, removing repetitions, and absorbing $\top$ and $\perp$). Normalization removes this irrelevant growth and makes the formula easier to read and interpret; as a side effect, it also exposes more top-level conjuncts to $\text{split}_{\wedge}$. Note that this normal form is deliberately syntactic, since it does not perform a semantic equivalence check. Language-equivalent residuals may therefore remain distinct, and $|cl(\varphi)|$ is in general larger than the number of states of the minimal DFA.

Now, we can prove that the repaired construction of RuleRunner is sound and complete.

Theorem 4. *Let φ be an LTL_f formula and let $RR_{\varphi}^{\text{prog}}$ be the progression-based RuleRunner monitor constructed as above. Then, the monitor returns SATISFY iff $\sigma \models \varphi$. Moreover, it is prefix-sound.*

The cost of the repair. One may wonder whether the new RuleRunner construction comes with a cost. It does, and where the cost is paid depends on how the monitor is realized.

In the *eager* realization, $\mathcal{R}_{\varphi}$ and its syntactic closure $\text{sub}(\mathcal{R}_{\varphi}) = \bigcup_{\chi \in \mathcal{R}_{\varphi}} \text{sub}(\chi)$ are computed once, and all corresponding evaluation and progression rules are compiled into a fixed finite rule system. Using the CILP translation, we can then map it to a recurrent network like for the original RuleRunner, either as a single flat or a structured network. The price is paid mainly in the recurrent state: it has one register per root, i.e. $|\mathcal{R}_{\varphi}|$ registers instead of the $|\text{sub}(\varphi)|$ of the original construction, and $|\mathcal{R}_{\varphi}|$ can be exponential in the worst case. Reactivation rules are also no longer per-operator templates: since $\text{prog}(\chi, \sigma_t)$ depends on the atoms $\mathcal{P}_{\chi}$ occurring in χ , the rules of a root encode a guarded transition over valuations of $\mathcal{P}_{\chi}$, exactly as the edges of a symbolic automaton do (see Sec. 3). Finally, exact early verdicts require deciding $\rho_t \equiv \top$ and $\rho_t \equiv \perp$, and similarly to what happens in automata-based monitors, we can enumerate the reachable sets of roots at compilation time and classifying them by reachability. This is optional: without it the monitor halts we the whole trace is consumed, which is still sound, but may emit a verdict later than necessary.

In the *lazy* realization, only the closure $\text{sub}(\rho_t)$ of the current residual is instantiated, one time point at a time. None of the axes above is enumerated: the registers are those of the parse tree of ρ_t , and the progression is computed only for that particular residual against the current observation, not expanded over $2^{|\mathcal{P}|}$. The cost per time step is therefore linear in the size of ρ_t , which, however, can be larger than φ . The cost here is that we cannot encode the monitor as a fixed recurrent network, since the rule set is no longer fixed, but dynamically generated. Instead, we can use a sequence of distinct feedforward neural networks, one per time step. This means that cross-trace batching is lost, since traces in a batch hold different residuals and share no weight matrix.

The counterexample revisited. Consider again the formula $\varphi_2 = F(a \wedge Xb)$ and the two traces $\sigma_A = (\{a\}, \emptyset, \{b\})$ and $\sigma_B = (\emptyset, \{a\}, \{b\})$. The residual formulas before each time step are:

$$\begin{array}{cccc}
\rho_0 & \xrightarrow{\sigma_0} \rho_1 & \xrightarrow{\sigma_1} \rho_2 & \xrightarrow{\sigma_2} \rho_3 \\
\sigma_A : F(a \wedge Xb) & \xrightarrow{\{a\}} (b \wedge F\top) \vee F(a \wedge Xb) & \xrightarrow{\emptyset} F(a \wedge Xb) & \xrightarrow{\{b\}} F(a \wedge Xb) \\
\sigma_B : F(a \wedge Xb) & \xrightarrow{\emptyset} F(a \wedge Xb) & \xrightarrow{\{a\}} (b \wedge F\top) \vee F(a \wedge Xb) & \xrightarrow{\{b\}} \top
\end{array}$$

For trace σ_A , the residual $(b \wedge F\top) \vee F(a \wedge Xb)$ appears after the first time step. The marker $F\top$ records that the pending b obligation requires another cell. At the second time step, b is false, so the disjunct disappears and the residual becomes $F(a \wedge Xb)$ again. After the last time step the same eventuality remains, and it is false on the empty suffix. Hence the verdict is VIOLATE, as expected. For trace σ_B , the residual $(b \wedge F\top) \vee F(a \wedge Xb)$ appears one step later, immediately before the time step containing b . At the last time step, b holds and $F\top$ progresses to true because that cell exists. Hence the residual becomes $\top$ and the verdict is SATISFY.

This is exactly the information lost by the original construction. The repaired monitor does not try to store two incompatible truth evaluations in the single register $[Xb]$. Instead, it stores the residual formula $(b \wedge F\top) \vee F(a \wedge Xb)$, and it occurs at different time points in the two traces.

5 Theoretical Comparison

5.1 The RuleRunner Ladder

Each monitoring paradigm has some structural weaknesses, so that no one paradigm dominates over the others:

- **Symbolic DFA: state-space blow-up.** The LTL_f -to-DFA translation is doubly exponential in the worst case, so the compiled automaton (and hence the compile-time cost and memory) can explode.
- **DeepDFA: explicit transition cost.** The transition tensor is indexed by the full alphabet $2^{|\mathcal{P}|}$. A factored backend can replace that unconditional tensor by disjoint guard cubes, which is often compact but remains exponential in the worst case, and every realization retains the $|Q|^2$ matrix update (see Section 3.3).
- **RuleRunner: temporal-instance conflation.** The one-register-per-subformula state can merge evaluations of the same subformula originating at different temporal offsets, so it can return wrong verdicts (Section ??). Bounded-future instances admit a fixed pipeline repair (Section 4.2), and full LTL_f completeness is recovered through the residual vocabulary of progression (Section 4.3).

Note that the state-space blow-up is not exclusive to the symbolic paradigm: DeepDFA is compiled from the *same* minimal DFA, so it inherits $|Q|$, and suffers the same doubly-exponential worst case. Second, the symbolic and automata-based heels are duals: a DFA transition function is a table with $|Q| \cdot 2^{|\mathcal{P}|}$ entries, and each paradigm compresses a different axis of it, paying on the axis it leaves explicit. The symbolic monitor compresses the symbol axis with Boolean guards and keeps an explicit state, so its cost driver is $|Q|$ and it can never suffer an alphabet blow-up; dense DeepDFA

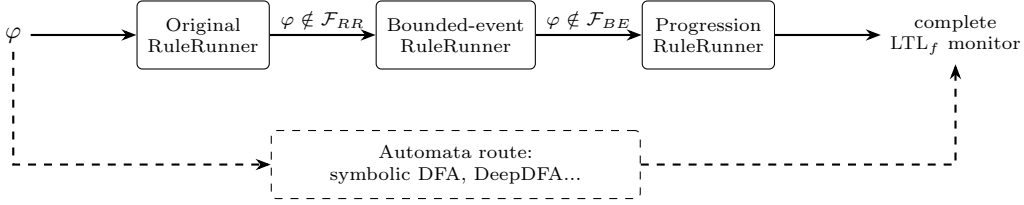


Figure 2: The construction ladder. A formula enters at the cheapest rung; a failed admission test moves it up.

compresses the state axis into a uniform batchable matrix product and therefore needs one matrix per symbol. Neither weakness can be swapped onto the other paradigm without discarding the representational choice that defines it.

The three different constructions of RuleRunner we identified should not be seen as competing alternatives, but as a ordered ladder. Each one is admitted by an explicit decidable test, builds a strictly larger compile-time object than the one below it, and buys a strictly larger guarantee. The rule system of the original construction is obtained by instantiating per-operator templates over $sub(\varphi)$ and is therefore linear in the size of the formula, the bounded-event pipeline adds a fixed window transducer and event circuit, and the eager progression construction enumerates residual roots rather than automaton states. Each LTL_f formula is admitted by one of the three rungs, and the ladder can be climbed until a rung is found that admits it. The procedure is summarized in Figure 2.

As already mentioned, the original construction and the bounded-event one pay a cost that is linear in the size of the formula and of the window, while the progression construction enumerates residual roots and may be exponential in the worst case. The automata-based monitors are doubly exponential in $|\varphi|$ in the worst case. Intuitively, it is therefore preferable to rely on RuleRunner, with the automata route being preferred only when the compile-time cost is amortized over a long-lived stream.

5.2 Capability Matrix

The heels above say where each paradigm breaks down. Table 1 states the complementary question: what each construction can do at all. Two readings of it matter for the rest of the paper. First, on crisp traces the exactness columns are almost uniformly filled, which is the interchangeability result restated: the choice between these monitors is not a choice about correctness, but about the cost of obtaining them and about the capabilities they expose beyond a crisp verdict. Second, the two groups of columns separate cleanly. Everything the rule-based lineage offers that the automata route does not concerns *compilation and state addressing*; everything the tensorized route offers that the rule-based lineage does not concerns *the type of the input and the differentiability of the computation*.

As monitors of crisp traces, the completeness-restored RuleRunner and fixed DeepDFA are the same semantically: the progression construction and the minimal DFA recognize the same LTL_f language (see Section 4.3). The reason to keep both lineages is what each could expose beyond crisp monitoring: a future learned DeepDFA would parameterize transitions between opaque automaton states, while a future rule-based learner could attach parameters to syntactically-indexed rules.

Construction	Semantics			Compilation		Neuro-symbolic affordances		
	All LTL_f	Exact final	Exact online	No automaton	Fixed network	Soft inputs	Differ-entiable	Syntax-indexed
Symbolic DFA	✓	✓	✓	–	–	–	–	–
DeepDFA	✓	✓	✓	–	✓	✓	✓	–
RuleRunner	–	✓ ^a	✓ ^a	✓	✓	–	✓	✓
Bounded-event	–	✓ ^b	–	✓	✓	–	✓	✓
Bounded-event, exact-online	–	✓ ^b	✓	✓	✓	–	✓	✓
Progression, eager	✓	✓	✓	✓ ^c	✓	–	✓	✓ ^d
Progression, lazy	✓	✓	–	✓	–	–	✓	✓ ^d

^a On the admitted class $\mathcal{F}_{RR}$ only.

^b On the admitted class $\mathcal{F}_{BE}$ only.

^c It does not build an automaton, but still builds all possible residual formulas by progression.

^d Indexed by residual formulas rather than by subformulas of φ .

Table 1: Features matrix. “Fixed network” means the compiled monitor is a single recurrent network with weights independent of the trace, which is what makes cross-trace batching possible; “syntax-indexed” means a register of the recurrent state names the subformula whose obligation it carries, so a verdict exhibits a witness.

These are two hypotheses for the gradient-based specification adaptation deferred to future work (Section 7), not capabilities evaluated here.

6 Experimental Comparison

7 Conclusion and Future Work

References

- [1] Oliver Arafat, Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification revisited. 2005.
- [2] Andreas Bauer, Martin Leucker, and Christian Schallhart. Monitoring of real-time properties. In *International Conference on Foundations of Software Technology and Theoretical Computer Science*, pages 260–272. Springer, 2006.
- [3] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Trans. Softw. Eng. Methodol.*, 20(4):14:1–14:64, 2011.
- [4] Alberto Camacho, Meghyn Bienvenu, and Sheila A. McIlraith. Towards a unified view of AI planning and reactive synthesis. In *ICAPS*, pages 58–67. AAAI Press, 2019.

- [5] Giuseppe De Giacomo, Antonio Di Stasio, Francesco Fuggitti, Sasha Rubin, et al. Pure-past linear temporal and dynamic logic on finite traces. In *IJCAI*, pages 4959–4965, 2020.
- [6] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In Francesca Rossi, editor, *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, pages 854–860. IJCAI/AAAI, 2013.
- [7] Giuseppe De Giacomo and Moshe Y Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Ijcai*, volume 13, pages 854–860, 2013.
- [8] Giuseppe De Giacomo and Moshe Y Vardi. Synthesis for LTL and LDL on finite traces. In *IJCAI*, pages 1558–1564. AAAI Press, 2015.
- [9] Cindy Eisner, Dana Fisman, John Havlicek, Yoad Lustig, Anthony McIsaac, and David Van Campenhout. Reasoning with temporal logic on truncated paths. In *International conference on computer aided verification*, pages 27–39. Springer, 2003.
- [10] Michael J Fischer and Richard E Ladner. Propositional dynamic logic of regular programs. *Journal of computer and system sciences*, 18(2):194–211, 1979.
- [11] Luca Geatti, Alessandro Gianola, Nicola Gigante, et al. Linear temporal logic modulo theories over finite traces. In *IJCAI*, volume 22, pages 2641–2647, 2022.
- [12] David Harel, Dexter Kozen, and Jerzy Tiuryn. Dynamic logic. In *Handbook of philosophical logic*, pages 99–217. Springer, 2000.
- [13] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. Introduction to automata theory, languages, and computation. *Acm Sigact News*, 32(1):60–65, 2001.
- [14] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [15] Bakhadyr Khoussainov and Anil Nerode. *Automata theory and its applications*. Springer Science & Business Media, 2012.
- [16] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. The glory of the past. In *Workshop on Logic of Programs*, pages 196–218. Springer, 1985.
- [17] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [18] Zohar Manna and Amir Pnueli. *Temporal Verification of Reactive Systems*, volume 1. Springer-Verlag, 1995.
- [19] Axel Mezini, Elena Umili, Ivan Donadello, Fabrizio Maria Maggi, Matteo Mancanelli, and Fabio Patrizi. Neuro-symbolic predictive process monitoring. *Information Systems*, 141:102762, 2026.

- [20] Alan Perotti, Guido Boella, and Artur d’Avila Garcez. Runtime verification through forward chaining. In *Runtime Verification: 6th International Conference, RV 2015, Vienna, Austria, September 22-25, 2015. Proceedings*, pages 185–200. Springer, 2015.
- [21] Alan Perotti, Artur d’Avila Garcez, and Guido Boella. Neural networks for runtime verification. In *2014 International Joint Conference on Neural Networks (IJCNN)*, pages 2637–2644. IEEE, 2014.
- [22] Alan Perotti, Artur d’Avila Garcez, and Guido Boella. Neural-symbolic monitoring and adaptation. In *2015 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2015.
- [23] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pages 46–57. IEEE Computer Society, 1977.
- [24] Elena Umili and Roberto Capobianco. Deepdfa: Automata learning through neural probabilistic relaxations. In *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, pages 1051–1058, 2024.
- [25] Moshe Y Vardi. The rise and fall of linear temporal logic. *Proceedings of the 2nd International Symposium on Games, Automata, Logics, and Formal Verification (11)*, 2011.

A Proofs

Theorem 1. Let θ be a formula in $\mathcal{F}_{RR}$ and σ a non-empty finite trace. Let RR_θ be the deterministic finite-state monitor induced by RuleRunner rules. Then RR_θ returns SATISFY iff $\sigma \models \theta$. Moreover, RR_θ is prefix-sound: if it outputs SATISFY (resp. VIOLATE) after reading the prefix $\sigma_0 \dots \sigma_t$ of σ , then $\sigma' \models \theta$ (resp. $\sigma' \not\models \theta$) for every trace σ' sharing the same prefix.

Proof. In the following, given a time step t , we write $v_t(\theta) \in \{T, F, ?\}$ for the truth value assigned to the $[\theta]$ by the evaluation fixpoint of time step t , ignoring the mode qualifier. We first show that, if the initial activation of a propositional formula π is in the state at the beginning of time step t , then the evaluation fixpoint of that cell assigns to every subformula of π exactly one truth value, and $v_t(\pi) = T$ iff $\sigma, t \models \pi$. In particular, a propositional formula is never undecided. The proof is by structural induction on π .

For $\pi = \top$ (resp. $\pi = \perp$) the single rule $R[\top] \rightarrow [\top]T$ (resp. $R[\perp] \rightarrow [\perp]F$) fires. For $\pi = p$, the observation p belongs to the time step iff $p \in \sigma_t$, so exactly one of $R[p], p \rightarrow [p]T$ and $R[p], \neg p \rightarrow [p]F$ fires. For $\pi = \neg\pi'$ the negation rule is the dual of the rules for π' . Finally, for $\pi = \pi_1 \wedge \pi_2$, both arguments are definite by inductive hypothesis, hence the truth table is exactly that of classical conjunction; the rows consuming a $?$ are not enabled, and the modes L and R are unreachable, because $R[\pi]L$ and $R[\pi]R$ are produced only by the reactivation of $[\pi]?L$ and $[\pi]?R$.

By this argument, we know that a propositional formula which is instantiated at time step t is always evaluated to a definite value within the same time step t , and it is discarded, together with every other truth register, at the end of that time step. Note that X , W , and U reinstalls only a propositional formula. Hence all the instances of a propositional subformula that are live at time step t , possibly created by different temporal operators, refer to the same time point t and agree on the value they write in the shared register. This is exactly the property that fails in Section ??.

Now, we now prove the following statement by structural induction on $\theta \in \mathcal{F}_{RR}$. Let σ be a non-empty trace and consider the run of the RuleRunner system of θ started from its initial activation at time step 0. Then:

- (i) at every cell t at which the node of θ is active, the register $[\theta]$ receives exactly one value $v_t(\theta)$;
- (ii) if $v_t(\theta) \neq ?$, then $v_t(\theta) = T$ iff $\sigma, 0 \models \theta$;
- (iii) if $v_t(\theta) = ?$ and $t < last$, the node of θ is active again at cell $t + 1$ with its mode recorded, and so are the arguments it has to read there;
- (iv) if $v_t(\theta) = ?$ for every $t \leq last$, then the END resolution of $[\theta]$ returns true iff $\sigma, 0 \models \theta$.

(Base case.)

- $\theta = \pi$. By the propositional case, $v_0(\pi)$ is definite and $v_0(\pi) = T$ iff $\sigma, 0 \models \pi$, which gives (i) and (ii); (iii) and (iv) are vacuous, and the node is not active after cell 0.
- $\theta = X\pi$. The rule $R[X\pi] \rightarrow [X\pi]?$ fires and no other rule of the node is enabled, so $v_0(X\pi) = ?$ and (i) holds, while (ii) is vacuous. If $0 = last$, then $\sigma, 0 \not\models X\pi$, and the END resolution returns false, which gives (iv). If $0 < last$, the reactivation of the unqualified $[X\pi]?$ installs $R[X\pi]M$ together with the initial activation of π , which gives (iii). By the propositional case $v_1(\pi) = T$ iff $\sigma, 1 \models \pi$, and $R[X\pi]M, [\pi]V \rightarrow [X\pi]V$, so $v_1(X\pi)$ is definite and it is true iff $\sigma, 0 \models X\pi$: this makes (iv) vacuous.

- $\theta = W\pi$. Same as the previous case, except that the END resolution of the unqualified $[W\pi]?$ returns true, which is correct.
- $\theta = \pi_1 \mathcal{U} \pi_2$. The node is activated at time step 0 in mode A , together with the initial activations of both arguments, and its reactivation re-installs both of them at every time step. Since by the propositional case the arguments are always definite, $?$ is never returned as truth value and the modes B , L and R are unreachable for the whole run. Hence, we can have $v_t(\pi_1 \mathcal{U} \pi_2)$ equals true if $\sigma, t \models \pi_2$, false if $\sigma, t \not\models \pi_2$ and $\sigma, t \not\models \pi_1$, and $?A$ otherwise. This gives (i) and (iii).

We prove by induction on t the invariant: *the node is active at cell t iff $\sigma, k \models \pi_1$ and $\sigma, k \not\models \pi_2$ for every $k < t$* . It holds trivially at $t = 0$, and it is preserved because the node is active at cell $t + 1$ only if $v_t = ?A$, i.e. only if π_1 holds and π_2 fails at t . Under the invariant, (ii) follows: if $v_t = T$ then $\sigma, t \models \pi_2$ and π_1 holds at every $k < t$, so t witnesses $\sigma, 0 \models \pi_1 \mathcal{U} \pi_2$; if $v_t = F$ then no $j < t$ witnesses the until by the invariant, $j = t$ fails because π_2 does, and no $j > t$ can witness it because π_1 fails at t , hence $\sigma, 0 \not\models \pi_1 \mathcal{U} \pi_2$. Finally, (iv): if the node is undecided at every cell, then by the invariant π_2 fails everywhere up to *last*, so $\sigma, 0 \not\models \pi_1 \mathcal{U} \pi_2$; and the END resolution of an until returns the END value of its right argument, which is the definite $[\pi_2]F$ recorded at cell *last*.

(Inductive case.)

- $\theta = \neg\theta'$. The negation rules map T to F , F to T and $?$ to $?$, so (i) holds and $v_t(\theta) \neq ?$ iff $v_t(\theta') \neq ?$, in which case (ii) follows from the inductive hypothesis on θ' . If $v_t(\theta') = ?$, then $[\neg\theta']? \rightarrow R[\neg\theta']$ carries the wrapper while θ' reactivates itself through its own rules, which together with the inductive hypothesis (iii) gives (iii). Likewise, the END resolution of $\neg\theta'$ is the complement of the END resolution of θ' , so (iv) follows from the inductive hypothesis (iv).
- $\theta = \theta_1 \wedge \theta_2$. Consider the first cell t at which one of the two arguments becomes definite; before t both are undecided, the row $(?, ?)$ of the mode- B table returns $?B$, and both arguments are carried and reactivate through their own rules by the inductive hypothesis (iii). If no such cell exists, (iv) holds because the END resolution in mode B recurses on both arguments, which are correct by the inductive hypothesis (iv), and the conjunction of their END values is the value of θ at time 0. Otherwise, we distinguish three cases at cell t . If both arguments are definite, the mode- B table returns their classical conjunction, and (ii) follows from the inductive hypothesis (ii) applied to both. If exactly one is definite and its value is F — say $v_t(\theta_1) = F$ — the row $(F, ?)$ returns F , and by the inductive hypothesis (ii) $\sigma, 0 \not\models \theta_1$, hence $\sigma, 0 \not\models \theta$ and (ii) holds. If exactly one is definite and its value is T — say $v_t(\theta_1) = T$ — the row $(T, ?)$ returns $?R$, so from cell $t + 1$ on only $R[\theta]R$ is carried and θ_1 is dropped from monitoring; dropping it is correct, because by the inductive hypothesis (ii) $\sigma, 0 \models \theta_1$ permanently, and θ_1 's own subsystem is not reactivated, its register being definite. The mode- R rules replicate the mode- B column at the pin value T of the left argument, so $v_{t'}(\theta) = v_{t'}(\theta_2)$ at every cell $t' \geq t$, and (ii), (iii) follow from the inductive hypothesis on θ_2 together with $\sigma, 0 \models \theta$ iff $\sigma, 0 \models \theta_2$. For (iv), the END resolution in mode R pins the left argument to T , which is the very value it was assigned at cell t , and recurses on θ_2 , so the claim follows from the inductive hypothesis (iv). The case in which θ_2 is the argument that settles is symmetric, with $?L$ in place of $?R$ and the right argument pinned to T .

In summary, if the value of θ becomes definite at some time step $t \leq last$, the monitor halts there and, by (ii), its verdict agrees with $\sigma \models \theta$ (this ensures also prefix soundness). Otherwise the register is undecided at every cell, and by (iv) the END resolution returns the correct value. In both cases RR_θ returns SATISFY iff $\sigma \models \theta$. $\square$

Corollary 1. If $\varphi \in \mathcal{F}_{RR}$, then $\varphi \in \mathcal{C}_{RR}$.

Proof. It follows directly by Theorem 1. $\square$

Lemma 1. Let φ be a formula with $h(\varphi) = h < \infty$. Let $l = \min(t + h, last)$ and $w = \sigma[t..l]$. For every non-empty trace σ and every position $t \leq last$,

$$\sigma, t \models \varphi \text{ if and only if } w, 0 \models \varphi$$

Proof. The proof is by structural induction on φ .

If φ is $\top$, $\perp$ or an atom, the statement is trivially satisfied.

If φ is $\neg\varphi'$, we have $h' = h(\varphi') \leq h$. It is easy to see that $\min(t + h', last) = \min(t + h', l) = t + \min(h', l - t)$. The inductive hypothesis applied to σ at t reduces φ' to the window $\sigma[t, \min(t + h', last)]$, and applied to w at 0 it reduces it to $w[0, \min(h', l - t)]$. Since φ' has the same value in σ at t and in w at 0, so does φ . If φ is $\varphi_1 \circ \varphi_2$, a similar argument as above applies.

If φ is $X\varphi'$, we have $h' = h(\varphi')$ so that $h = 1 + h'$. When $t = last$ we have $l = t$ and w has a single position, so both $\sigma, t \models X\varphi'$ and $w, 0 \models X\varphi'$ are false. When $t < last$ we have $l \geq t + 1$, so both traces have a successor position and the two sides amount to $\sigma, t + 1 \models \varphi'$ and $w, 1 \models \varphi'$. The inductive hypothesis reduces them to $\sigma[t + 1, \min(t + 1 + h', last)]$ and $w[1, \min(1 + h', l - t)]$, which is the same trace. The case $\varphi = W\varphi'$ is the same, except that a single-position w makes both sides true. $\square$

Theorem 2. Let $\alpha = sk(\varphi)$. If RR_α is language-equivalent to A_α , then for every finite trace σ the composition of the bounded-event transducer and RR_α returns SATISFY iff $\sigma \models \varphi$. Moreover, if RR_α is prefix-sound, then so is the composition.

Proof. The empty trace is settled by the separately computed boundary value, since it has no position at which an event could be emitted, so let σ be non-empty. We prove the claim in two steps.

(The transducer emits the derived trace.) Consider a position p emitted while the trace is being read, so that its buffer is $\sigma[p..p + H]$ with $p + H \leq last$. For every island φ_i , Lemma 1 applied to σ at p reduces $\sigma, p \models \varphi_i$ to the window $\sigma[p..p + h(\varphi_i)]$, and applied to the buffer at 0 it reduces the transducer's own evaluation to the same window, because $h(\varphi_i) \leq H$. The two values therefore coincide. Consider now a position p emitted during the flush: its buffer is exactly the suffix $\sigma[p..last]$, and evaluating a formula at position 0 of a suffix is evaluating it at p in σ , the truncated readings of X and W being the end-of-trace semantics of LTL_f . In both cases the emitted cell also carries σ_p unchanged, and the cells are emitted in increasing order of p , one per input cell. Hence the transducer outputs exactly $\hat{\sigma}$.

(Substitution.) We show that $\sigma, t \models \varphi$ iff $\hat{\sigma}, t \models \alpha$ for every position t , by structural induction on φ following the rewriting that produces α . If the node is one of the islands φ_i , it is replaced by e_i , and $e_i \in \hat{\sigma}_t$ iff $\sigma, t \models \varphi_i$ by the definition of the derived trace. If the node is an atom, it is left unchanged, and $\hat{\sigma}_t \cap \mathcal{P} = \sigma_t$. Otherwise the node is unbounded, the rewriting preserves its operator

and recurses on its arguments, and the inductive hypothesis applies to them; the positions at which they are evaluated are the same in the two traces, because $\hat{\sigma}$ and σ have the same length, hence the same *last*.

Taking $t = 0$, the skeleton monitor consumes $\hat{\sigma}$ and, by language equivalence, returns SATISFY iff $\hat{\sigma} \models \alpha$, that is iff $\sigma \models \varphi$.

Assume now that RR_α is also prefix-sound, and suppose that it emits a definite verdict after the transducer has read $\sigma_0, \dots, \sigma_t$. At that point the transducer has emitted the prefix $\hat{\sigma}_0, \dots, \hat{\sigma}_{t-H}$ of the derived trace, which is empty when $t < H$. Prefix soundness makes that verdict correct for every trace over $\mathcal{P} \cup \{e_1, \dots, e_m\}$ extending the emitted prefix. Now, every non-empty trace σ' extending $\sigma_0, \dots, \sigma_t$ derives a trace $\hat{\sigma}'$ that extends that prefix: each emitted position $p \leq t - H$ has its whole window $\sigma[p..p + H]$ inside the part already read, so by Lemma 1 it receives the same value in σ' , whether the extension is proper or σ' stops at t and p is flushed. The realizable derived continuations are therefore a subset of the continuations quantified over by prefix soundness, and by substitution the verdict agrees with $\sigma' \models \varphi$ for all of them. $\square$

Corollary 2. For every $\varphi \in \mathcal{F}_{BE}$, the bounded-event RuleRunner returns the correct final verdict, and it is prefix-sound.

Proof. By definition, if $\varphi \in \mathcal{F}_{BE}$ its skeleton $sk(\varphi)$ belongs to $\mathcal{F}_{RR}$. By Corollary 1, every formula in $\mathcal{F}_{RR}$ is in $\mathcal{C}_{RR}$, so $RR_{sk(\varphi)}$ is language-equivalent to $A_{sk(\varphi)}$; and by Theorem 1 it is also prefix-sound. Thus, we can conclude that the bounded-event RuleRunner returns SATISFY iff $\sigma \models \varphi$ and it is also prefix-sound. $\square$

Theorem 3. After every non-empty prefix, the extrapolated bounded-event monitor returns a three-valued label identical to that of the canonical DFA for the original formula.

Proof. By Theorem 2, $\mathcal{L}(P_\varphi) = \mathcal{L}(A_\varphi)$. In a deterministic automaton, the state reached after a prefix u is an accepting sink if and only if every continuation of u belongs to the recognised language, and a trap if and only if no continuation does; both conditions depend only on u and on the language, not on the automaton recognising it. Since P_φ and A_φ recognise the same language and read the same alphabet, the state reached by P_φ after u is an accepting sink, respectively a trap, exactly when the state reached by A_φ is. The extrapolated monitor returns the label of the former and the symbolic monitor of Section 2.3 returns the label of the latter, so the two labels coincide. $\square$

Theorem 4. Let φ be an LTL_f formula and let RR_φ^{prog} be the (eager) progression-based RuleRunner monitor constructed as above. Then, for every finite trace σ , the monitor returns SATISFY iff $\sigma \models \varphi$, and it returns VIOLATE iff $\sigma \not\models \varphi$. Moreover, any verdict returned before the last cell is sound for all finite continuations of the prefix read so far.

Proof. Let $\sigma = (\sigma_0, \dots, \sigma_{T-1})$ be a finite trace. For $0 \leq t \leq T$, let Σ_t be the active root set after the prefix $\sigma_0 \dots \sigma_{t-1}$ has been consumed (and thus before reading σ_t when $t < T$). Let

$$\rho_t = \bigwedge_{\chi \in \Sigma_t} \chi$$

be the residual it denotes. By construction,

$$\Sigma_0 = \text{split}_\wedge(\text{nf}(\varphi)),$$

and therefore ρ_0 is equivalent to φ .

We first use the defining property of progression. For every formula χ , every cell s , and every finite continuation τ , including the empty continuation ϵ ,

$$s \cdot \tau \models \chi \iff \tau \models \text{prog}(\chi, s). \quad (4)$$

This property follows by structural induction on χ . The atomic and Boolean cases are immediate from the definition of **prog**. For $X\chi$, $s \cdot \tau \models X\chi$ iff τ is non-empty and $\tau \models \chi$. Since $F\top$ holds exactly on non-empty finite traces, this is equivalent to $\tau \models \chi \wedge F\top$. Dually, $W\chi$ is true either when the suffix is empty or when χ holds at its first cell; $G\perp$ holds exactly on the empty trace, yielding $\text{prog}(W\chi, s) = \chi \vee G\perp$. For until, $s \cdot \tau \models \chi_1 U \chi_2$ iff either χ_2 holds at the current cell, or χ_1 holds at the current cell and $\chi_1 U \chi_2$ holds on the suffix; by the induction hypothesis this is equivalent to

$$\tau \models \text{prog}(\chi_2, s) \vee (\text{prog}(\chi_1, s) \wedge (\chi_1 U \chi_2)).$$

The derived operators inherit the property from their definitions.

We now prove the residual invariant:

$$\sigma_0 \cdots \sigma_{t-1} \cdot \tau \models \varphi \iff \tau \models \rho_t \quad (5)$$

for every $0 \leq t \leq T$ and every finite continuation τ .

For $t = 0$, the claim holds because ρ_0 is equivalent to φ . Assume it holds for some $t < T$. By the reactivation rule,

$$\rho_{t+1} \equiv \text{nf} \left(\bigwedge_{\chi \in \Sigma_t} \text{prog}(\chi, \sigma_t) \right).$$

Since $\rho_t = \bigwedge_{\chi \in \Sigma_t} \chi$ and progression distributes over conjunction up to logical equivalence, this is equivalent to

$$\rho_{t+1} \equiv \text{nf}(\text{prog}(\rho_t, \sigma_t)).$$

Therefore, for every continuation τ ,

$$\begin{aligned} \sigma_0 \cdots \sigma_t \cdot \tau \models \varphi &\iff \sigma_t \cdot \tau \models \rho_t \quad \text{by the induction hypothesis,} \\ &\iff \tau \models \text{prog}(\rho_t, \sigma_t) \quad \text{by Equation (4),} \\ &\iff \tau \models \rho_{t+1} \quad \text{because nf preserves semantics.} \end{aligned}$$

Thus the invariant holds at $t + 1$.

After consuming all T cells, repeated application of the invariant gives

$$\sigma \models \varphi \iff \epsilon \models \rho_T.$$

The final evaluation rules compute precisely whether the residual accepts the empty trace. Hence the monitor returns SATISFY exactly on traces satisfying φ , and VIOLATE otherwise.

Finally, the operational register construction realizes the symbolic transition used in the proof. At time t , the state $S_t = \text{sub}(\rho_t)$ contains every subformula needed to evaluate and progress the active roots. Since the eager monitor allocates registers for all formulas in $\text{sub}(cl(\varphi))$, every such support formula has a corresponding register, and the bottom-up computation can be carried out by finitely many rules. Only the active roots in Σ_t are reactivated, so support subformulas do not

acquire an independent temporal meaning. Therefore the rule-based monitor implements exactly the residual update used in the invariant.

If an early verdict is returned, the current residual is respectively valid or unsatisfiable over all finite continuations. By Equation (5), every completion of the prefix read so far then has the same truth value for φ , which proves soundness of early termination. $\square$